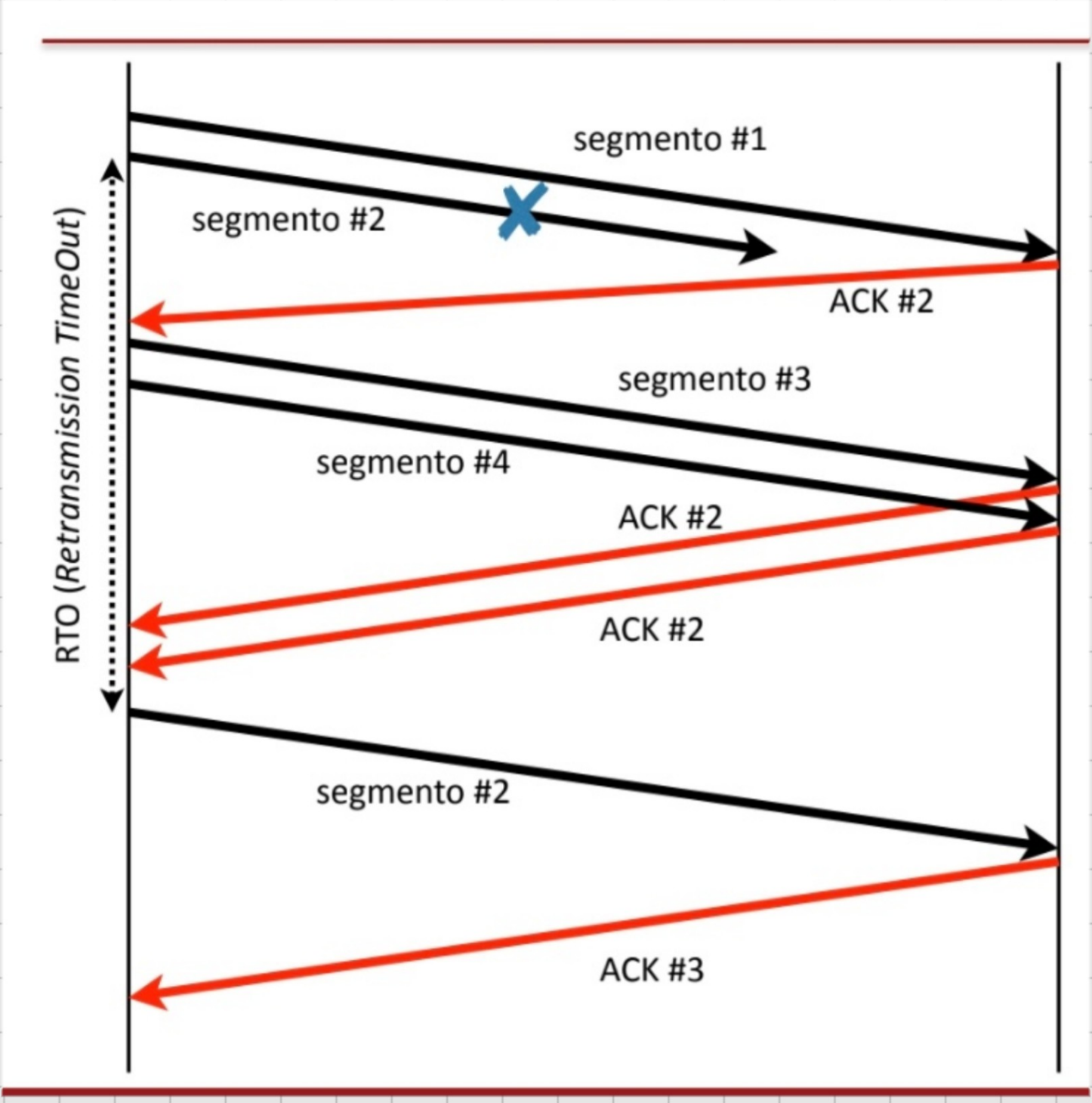
CONTROLLO DI ERRORE IN TCP



- ARQ ⟶ SLIDING WINDOW GO-BACK-N CON RISCONTRI POSITIVI
- I BYTE NELLA PARTE DATI SONO NUMERATI SEQUENZIALMENTE A PARTIRE DAL NUMERO SCELTO DURANTE IL THREE-WAY HANDSHAKING
- I SEGMENTI PORTANO UN ACK NUMBER (UTILIZZATO ANCHE DAL CONTROLLO DI FLUSSO/CONGESTIONE)

- QUANDO SI RICEVE UN SEGMENTO DATI VIENE INVIATO UN RISCONTRO RITARDATO DI UNA FRAZIONE DI SECONDO (DELAYED ACKNOWLEDGEMENT)

- RITRASMISSIONE PER MANCATA RICEZIONE DI UN ACK ENTRO IL TIMEOUT

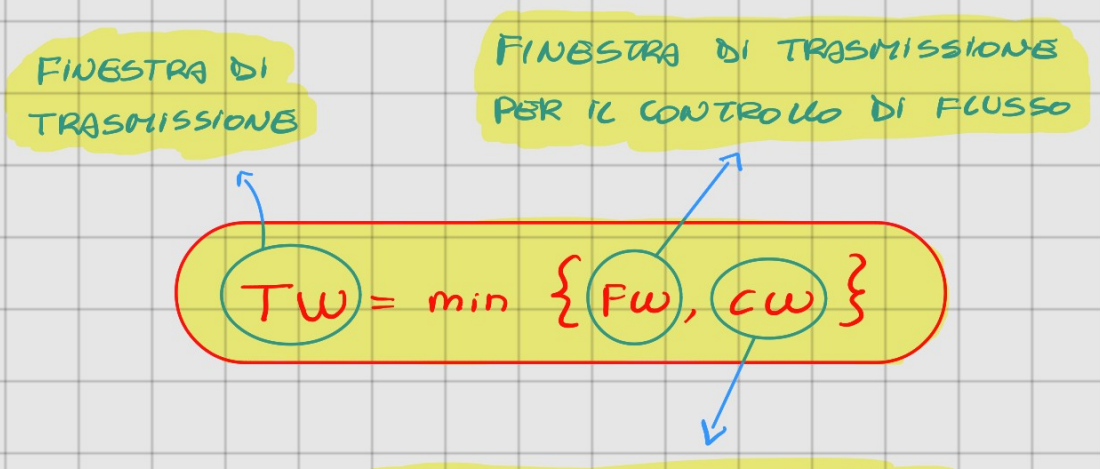
- SE UN SEGMENTO ARRIVA CON UN CHECKSUM NON VALIDO, IL SEGMENTO VIENE SCARTATO E SI ATTENDE CHE IL TRASMETTITORE VADA IN TIMEOUT PER RITRASMETTERE IL SEGMENTO

FINESTRA DI TRASMISSIONE

- TRASMISSION WINDOW (TW) DI DIMENSIONE VARIABILE

- RISCONTRO CON ACK NUMBER PARI A N SIGNIFICA CHE

  * SONO RISCONTRATI TUTTI I BYTE RICEVUTI FINO A QUELLO NUMERATO CON N-1
  * IL TRASMETTITORE È AUTORIZZATO A TRASMETTERE UN ULTERIORE NUMERO DI BYTE PARI ALLA FINESTRA DI TRASMISSIONE



FINESTRA DI TRASMISSIONE PER IL CONTROLLO DI CONGESTIONE

TIMEOUT DI RITRASMISSIONE

Il suo dimensionamento è un aspetto critico per le prestazioni

- se troppo piccolo alcuni segmenti in ritardo potrebbero essere considerati persie quindi ritrasmessi (conseguente aumento di congestioni e ritrasmissioni)
- se troppo grande la risposta ad un evento di perdita sarebbe troppo lenta (conseguente perdita di efficienza)

Il RTO è fissato in maniera adattativa misurando dinamicamente il **Round Trip Time (RTT),** cioè **il ritardo tra l'invio di un segmento e la ricezione del relativo ACK**

- RTT variable in base alle condizioni della rete
- Problemi nella misura del RTT:

1. l'emissione degli ACK da parte del ricevente può essere non immediata
2. se è stata effettuata una ritrasmissione è impossibile distinguere se l'ACK si riferisce alla trasmissione iniziale o alla ritrasmissione (soluzione nell'algoritmo di Karn)
3. lo stato di congestione della rete può cambiare molto rapidamente

DIMENSIONAMENTO DELL'RTO

- Il RTO è fissato con uno schema adattativo

$$\begin{aligned}
\overline{\tau}_k &= \frac{1}{k}\sum_{i=1}^{k}\tau_i \\
&= \frac{1}{k}\sum_{i=1}^{k-1}\tau_i + \frac{1}{k}\tau_k \\
&= \frac{k-1}{k-1}\frac{1}{k}\sum_{i=1}^{k-1}\tau_i + \frac{1}{k}\tau_k \\
&= \frac{k-1}{k}\underbrace{\frac{1}{k-1}\sum_{i=1}^{k-1}\tau_i}_{=\overline{\tau}_{k-1}} + \frac{1}{k}\tau_k \\
&= \frac{k-1}{k}\overline{\tau}_{k-1} + \frac{1}{k}\tau_k
\end{aligned}$$

RTT istantaneo misurato (possibilmente con errore) al segmento *i*

RTT medio misurato fino al segmento *i*

RTT ACK i τ_i $\{\tau_1, \tau_2, \ldots, \tau_K\}$

RTT MEDIO AL PACCHETTO K

$$\overline{\tau}_K = \frac{\tau_1+\tau_2+\ldots+\tau_K}{K} = \frac{\tau_1+\tau_2+\ldots+\tau_{K-1}}{K} + \frac{\tau_K}{K}$$

$$= \frac{1}{K}\sum_{i=1}^{K}\tau_1$$

$$= \frac{1}{K}\sum_{i=1}^{K-1}\tau_i + \frac{\tau_K}{K}$$

$$= \frac{K-1}{K-1}\frac{1}{K}\sum_{i=1}^{K-1}\tau_i + \frac{\tau_K}{K}$$

$$= \frac{K-1}{K}\frac{1}{K-1}\sum_{i=1}^{K-1}\tau_i + \frac{\tau_K}{K}$$

$$= \frac{K-1}{K}\overline{\tau}_{K-1} + \frac{1}{K}\tau_K$$

nella specifica originale di TCP la media pesata degli RTT è denominata *Smoothed Round Trip Estimate* (SRTT): l'idea è dare maggior peso alle stime recenti

$$\hat{\tau}_k = \alpha\overline{\tau}_{k-1} + (1-\alpha)\tau_k$$

$$\text{RTO}_k = \beta\hat{\tau}_k$$

parametro per scegliere un RTO maggiore dell'RTT misurato
- tipicamente tra 1.3 e 2
- il valore raccomandato è 2

parametro per il pesaggio
- tipicamente tra 0.8 e 0.9
- il valore raccomandato è 0.8

- IL METODO DI MISURA STANDARD DI SRTT NON È ADATTO A SITUAZIONI IN CUI LA VARIANZA DEL RITARDO DI RETE È VARIABILE
  - * QUANDO LA VARIANZA È BASSA, UNA SOVRASTIMA RTO PORTA AD ATTENDERE PIÙ TEMPO DEL DOVUTO
  - * QUANDO LA VARIAZIONE È ALTA, LA STIMA DI RTO PUÓ NON ESSERE COMUNQUE SUFFICIENTE AD EVITARE TRASMISSIONI INUTILI

- LA STIMA DELLA DEVIAZIONE STANDARD DELL'RTT È COMPLESSA E SI APPLICA L'ALGORITMO DI JACOBSON

$$\hat{\tau}_k = \hat{\tau}_{k-1} + g \cdot \varepsilon_k$$ ← valore consigliato del parametro: 1/8

$$\varepsilon_k = \tau_k - \hat{\tau}_{k-1}$$

$$\hat{\sigma}_{\tau,k} = (1-h) \cdot \hat{\sigma}_{\tau,k-1} + h \cdot |\varepsilon_k|$$ ← valore consigliato del parametro: 1/4

$$\text{RTO}_k = \hat{\tau}_k + f \cdot \hat{\sigma}_{\tau,k}$$ ← valore consigliato del parametro: 4

Certamente! Le formule nell'immagine che hai inviato sono relative al controllo degli errori e al controllo della congestione nel protocollo TCP. Queste formule sono utilizzate per calcolare il Timeout di Ritrasmissione (RTO). Ecco una spiegazione:

1. **Stima dell'Errore ((\hat{E}_k))**:

$$\hat{E}_k = T_k - \hat{T}_{k-1}$$

Questa formula calcola l'errore stimato sottraendo il tempo stimato di andata e ritorno (RTT) precedente dal tempo RTT attuale.

2. **Deviazione dell'Errore Stimato ((\sigma_{T,k}))**:

$$\sigma_{T,k} = (1 - h) \cdot \sigma_{T,k-1} + h \cdot | \hat{E}_k |$$

Questa calcola la deviazione dell'errore stimato, considerando una somma ponderata della deviazione precedente e del valore assoluto dell'errore stimato corrente.

3. **Calcolo del RTO ((RTO_k))**:

$$RTO_k = \hat{T}_k + f \cdot \sigma_{T,k}$$

Questa è utilizzata per calcolare il Timeout di Ritrasmissione, che è la somma del tempo RTT stimato e un fattore moltiplicato per la deviazione.

La gestione dell'RTO è fondamentale per garantire l'affidabilità del protocollo TCP, in quanto determina quando i pacchetti dovrebbero essere ritrasmessi in caso di perdita o ritardo.

1. **T_k:** Questo rappresenta il tempo di andata e ritorno (RTT) misurato per il pacchetto corrente

2. **T_k-1:** Questo è il tempo RTT stimato per il pacchetto precedente

3. **E_k:** Questo è l'errore stimato, calcolato come la differenza tra il tempo RTT attuale e il tempo RTT stimato precedente

4. **sigma{T,k}:** Questo rappresenta la deviazione standard dell'errore stimato. È calcolato considerando una combinazione ponderata della deviazione precedente e del valore assoluto dell'errore stimato corrente

5. RTO_k: Questo è il Timeout di Ritrasmissione (RTO), che determina quanto tempo dovrebbe passare prima di ritrasmettere un pacchetto in caso di perdita o ritardo. È la somma del tempo RTT stimato e di un fattore moltiplicato per la deviazione

CONTROLLO DI FLUSSO IN TCP

- OBIETTIVO: LIMITARE IL FLUSSO DEI DATI, IN BASE ALLE ESIGENZE DEL TERMINALE RICEVENTE, PRESCINDENDO DAL TRAFFICO PRESENTE NELLA RETE

  ↓

  * PERMETTE LA COMUNICAZIONE TRA TERMINALI DI DIMENSIONE E VELOCITÁ MOLTO DIVERSE TRA LORO SENZA PERDITE DI INFORMAZIONE

- HA IL COMPITO DI ASSICURARE CHE IL RITMO DI ARRIVO DEI PACCHETTI NON SUPERI LA CAPACITÁ DI MEMORIZZAZIONE ED ELABORAZIONE DELLA ENTITÁ RICEVENTE

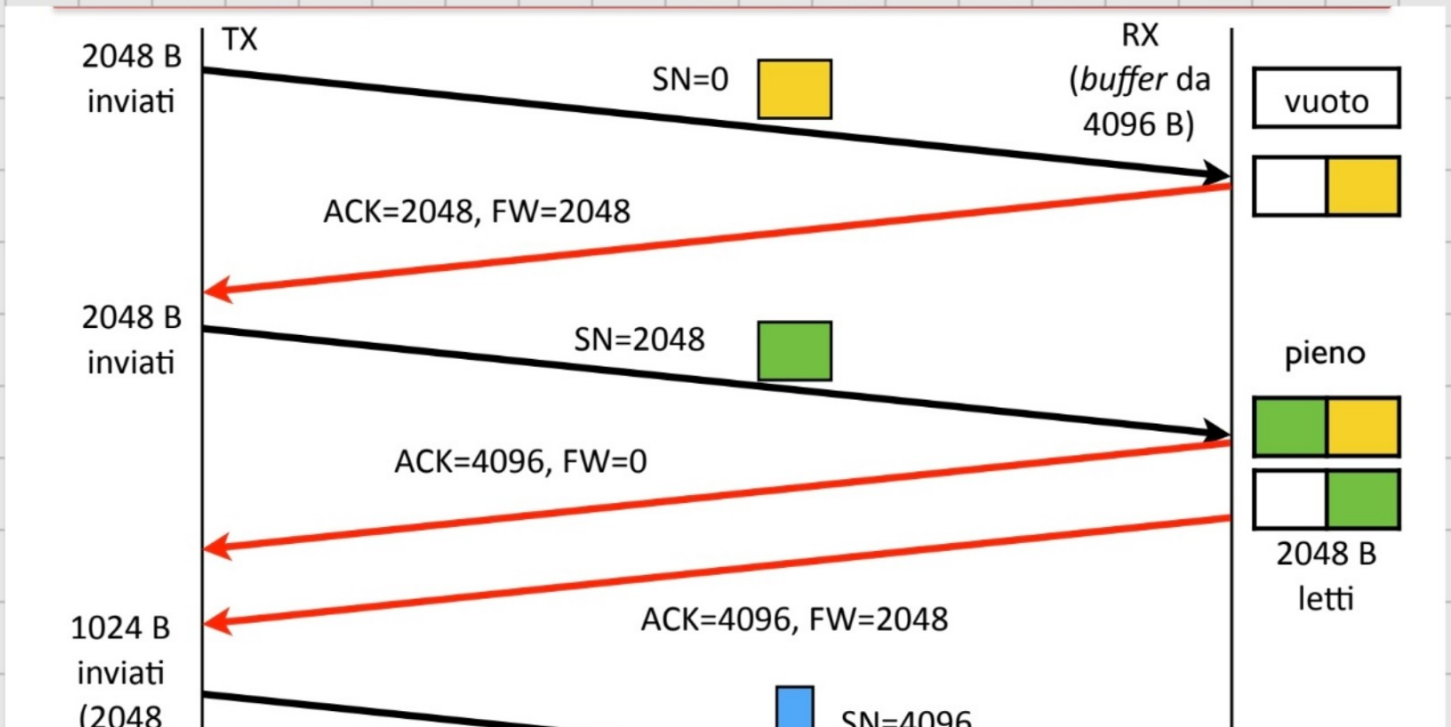
- Regolazione dell'emissione del flusso di dati di una entità e mittente da parte della entità ricevente
- Dipende dalle capacità di memorizzazione del ricevente (**buffer**) e dalla sua velocitàdielaborazioneesmaltimento
- Spesso viene realizzata in interazione con la funzione di recupero di errore e/o di controllo di congestione

Tecniche con messaggi di riscontro o indicazione da parte del ricevitore della dimensione della finestra di numeri di sequenza ammessidalricevitore

CONTROLLO DI FLUSSO IN TCP

- OPERA A LIVELLO DI OTTETTI
- SI BASA SU UNA FINESTRA SCORREVOLE DI AMPIEZZA VARIABILE
- I SEGMENTI CONTENGONO
  - * ACK NUMBER (UTILIZZATO PURE DAL CONTROLLO DI ERRORE)
  - * FLOW WINDOW (FW)
- UN RISCONTRO CON ACK NUMBER = N E FW = W SIGNIFICA CHE
  - * SONO RISCONTRATI TUTTI GLI OTTETTI RICEVUTI FINO A QUELLO NUMERATO CON N-1
  - * IL TRASMETTITORE È AUTORIZZATO A TRASMETTERE FINO AD ULTERIORI W BYTE, OVVERO FINO A QUELLO NUMERATO CON N+W-1

ESEMPIO: BUFFER DI 4096 B





1. Titolo e Scopo:

- Il buffer da 4096 B è una zona di memoria temporanea utilizzata per immagazzinare dati durante la comunicazione o l'elaborazione.

2. Passo 1: Invio dei Dati (TX):

- Inizia con 2048 B inviati
- Questi dati vengono trasmessi attraverso una connessione (ad esempio, una rete o un canale di comunicazione)
- Il buffer viene riempito con questi 2048 byte

3. Passo 2: Ricezione dei Dati (RX):

- Il ricevitore (RX) riceve i dati inviati
- Il numero di sequenza (SN) è **0**
- L'acknowledgment (ACK) è **2048**, e la finestra di flusso (FW) è **2048**
- Il buffer RX è **vuoto**

4. Passo 3: Invio di Altri 2048 B (TX):

- Vengono inviati **altri 2048 B**
- Il buffer TX è ancora in uso
- Il ricevitore (RX) ha ora **2048 B** nel buffer

5. Passo 4: Ricezione di Altri 2048 B (RX):

- Il ricevitore (RX) riceve altri 2048 byte
- Il numero di sequenza (SN) è ora 2048
- **L'acknowledgment** (ACK) è **4096**, e la finestra di flusso (FW) è **0** (nessun altro dato può essere inviato)
- Il buffer RX è ora pieno

6. Passo 5: Invio di Altri 1024 B (TX):

- Vengono inviati 1024 B (anche se potrebbero essere stati inviati fino a 2048 B)
- Il buffer TX è ancora in uso
- Il ricevitore (RX) ha ora **2048 B** nel buffer (dato letto)

7. Stato Finale del Buffer RX:

- Il buffer RX ha 2048 B letti
- È rappresentato da un quadrato blu nell'immagine

In breve, il buffer da 4096 B gestisce l'invio e la ricezione dei dati, garantendo che non vengano persi e ottimizzando le prestazioni

CONTROLLO DI CONGESTIONE IN TCP

* COSA VUOL DIRE CONTROLLO DI CONGESTIONE ?

- IL CONTROLLO DI CONGESTIONE HA CO SCOPO DI RECUPERARE EVENTUALI SITUAZIONI DI SOVRACCARICO (CONGESTIONE) DELLA RETE

  ↓

  * REGOLAZIONE DELL'EMISSIONE DEI DATI IN MODO DA RIDURRE IL

GRADO DI CONGESTIONE DELLA RETE

- TECNICHE
  - * ROUTING ADATTATIVO
  - * CONTROLLO DI AMMISSIONE DI CHIAMATA NEL CASO DI COMUNICAZIONE
  - * MECCANISMI CON FEEDBACK DEI NODI DELLA RETE
  - * MECCANISMI END-TO-END

DIFFICOLTÁ

- LA CONGESTIONE DIPENDE DAI NODI DI RETE (DI LIVELLO IP)
  - * TCP È UN PROTOCOLLO DI TRASPORTO CHE OPERA SOLO NEI NODI TERMINALI
  - * LA CONGESTIONE (SOVRACCARICO) PUÓ ESSERE RILEVATA DA ESTREMO AD ESTREMO

- IL PROTOCOLLO DI RETE IP NON POSSIEDE ALCUN MECCANISMO PER RIVELARE/CONTROLLARE LA CONGESTIONE
  - * TCP PUÓ SOLO RIVELARE E CONTROLLARE LA CONGESTIONE IN MODO INDIRETTO, MISURANDONE GLI EFFETTI A LIVELLO DI TRASPORTO
  - * LA CONOSCENZA DELLO STATO DELLA RETE DA PARTE DELLE ENTITÁ TCP È IMPERFETTA

- LE ENTITÁ TCP COINVOLTE SU CONNESSIONI DIVERSE (E SU HOST DIVERSI), SEBBENE CONDIVIDANO LA STESSA RETE E COMPETONO PER IL SUO UTILIZZO, NON COOPERANO DIRETTAMENTE

TRA LORO

Osservazioni

- In caso di congestione, il controllo di errore e di flusso a finestra proteggono implicitamente anche la rete
  - se la rete è congestionata arriveranno meno riscontri e quindi saranno emessi un numero minore di segmenti
  - il meccanismo adattativo di *timeout* evita ritrasmissioni che porterebbero ad un aumento della congestione invece che ad una sua diminuzione
- Il risultato è che il ritmo di emissione della sorgente tende ad adattarsi automaticamente al collo di bottiglia della rete
- La funzione di controllo di congestione interviene cercando di limitare ulteriormente il carico in modo da rimuovere lo stato di congestione
  - funziona se tutti gli *host* collaborano eseguendo la una procedura simile di controllo di congestione

CONTROLLO DI CONGESTIONE IN TCP

CAUSE DEI COLLI DI BOTTIGLIA

↓

- LIMITAZIONE DELLA BANDA NEI COLLEGAMENTI FISICI
- CONGESTIONE NEI ROUTER
- LIMITATA CAPACITÁ ELABORATIVA DEL RICEVITORE

IL CONTROLLO DI CONGESTIONE (SOVRACCARICO) DI TCP

↓

- NON È IN GRADO DI STABILIRE LA CAUSA SPECIFICA E LA CONTROMISURA PIÚ ADATTA
- UTILIZZA LA SCADENZA DEL RTO COME INDICATORE DI CONGESTIONE

IL CONTROLLO DI CONGESTIONE DEL TCP SI BASA SU UNA FINESTRA DI EMISSIONE (CONGESTION WINDOW, CW) DI DIMENSIONE CHE PUÓ VARIARE

↓

* INDICA IL NUMERO MASSIMO DI BYTE CHE POSSONO ESSERE EMESSI SENZA RISCONTRO
* FISSATO IL RTT, VARIANDO LA DIMENSIONE DELLA FINESTRA VARIA IL RITMO DI EMISSIONE

SLOW START

REGOLA L'EMISSIONE DEI SEGMENTI AD OGNI RIPARTENZA

↓

- ALL'INIZIO DELLA CONNESSIONE
- DOPO OGNI SITUAZIONE DI CONGESTIONE

HA LO SCOPO DI AUMENTARE IL RITMO DI EMISSIONE SINO AL RAGGIUNGIMENTO DEL LIMITE DI CONGESTIONE

↓

- PROVOCANDO COSÍ UNA NUOVA RIPARTENZA

SI CONSIDERA UNA CW CHE TENDE AD AUMENTARE

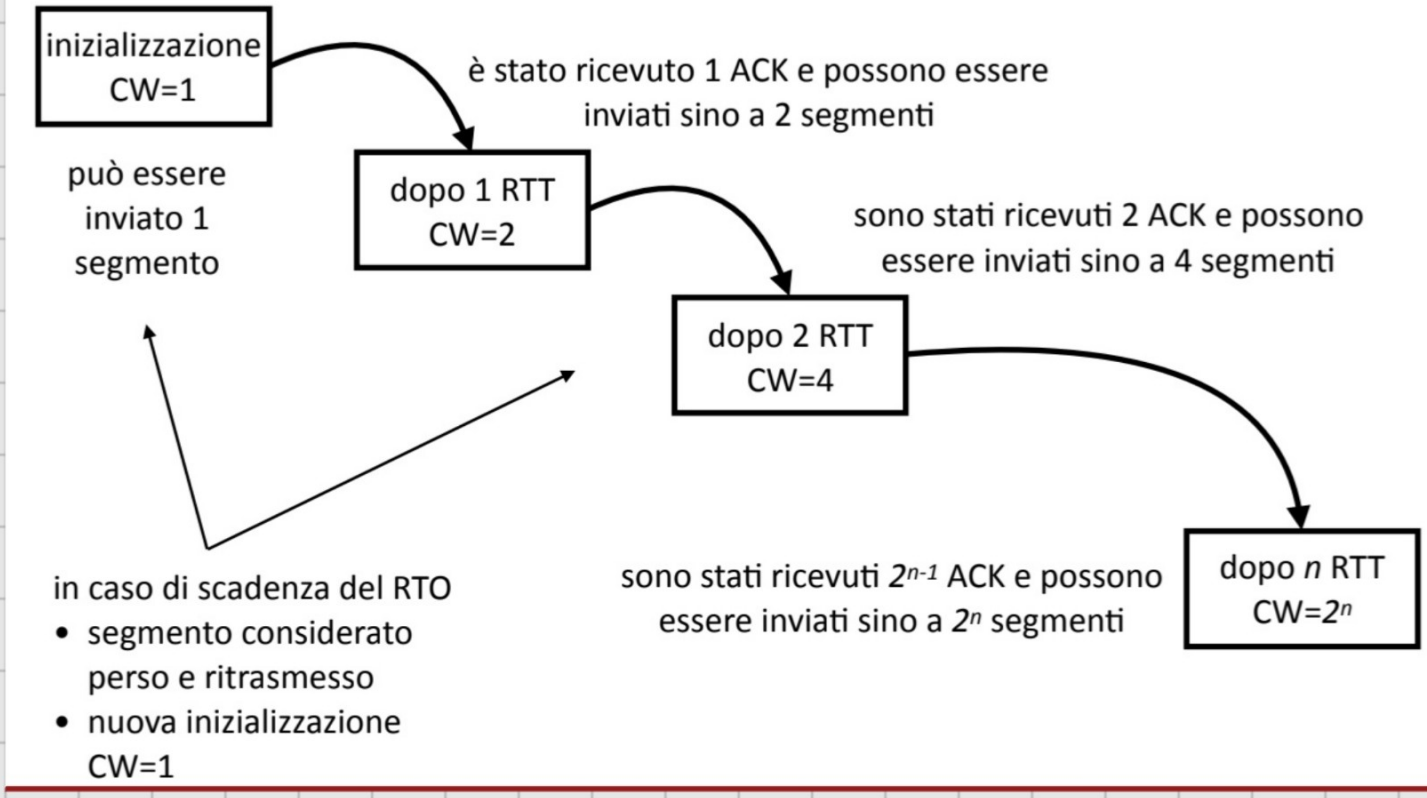
↓

- VARIA CON MULTIPLI DI MSS
- VIENE INCREMENTATA DI UN MSS PER OGNI SEGMENTO TRASMESSO E CONFERMATO

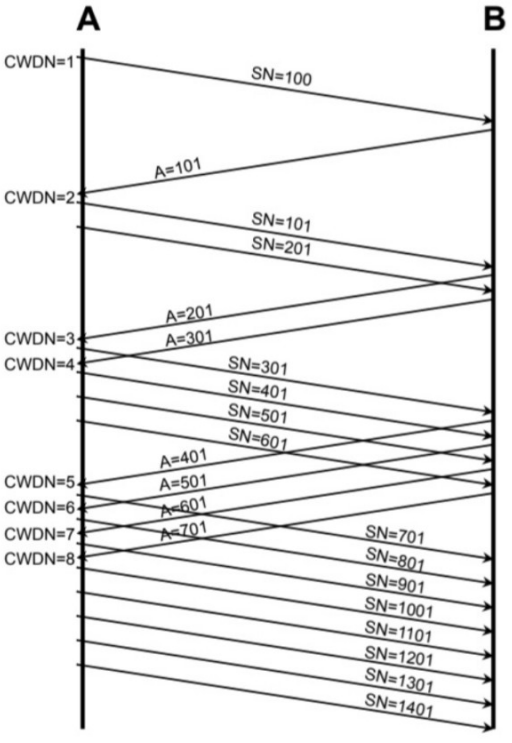
MASSIMO VALORE AMMESSO

$$CW = \min \{ MAx\ Win, CW + MSS \}$$

CRESCITA ESPONENZIALE DELLA CW



SLOW START



CONGESTION AVOIDANCE

- PROBLEMI DELLA PROCEDURA SLOW START

  ⇓

  * IN CASO DI CONGESTIONE SI RIPARTE DA CW = 1
  * L'INCREMENTO ESPONENZIALE È MOLTO AGGRESSIVO

!! IDEA: RIDURRE LA CRESCITA DELLA CW DOPO OGNI RIPARTENZA !!

- SI RIDUCE IL FLUSSO IN MODO CHE

  ⇓

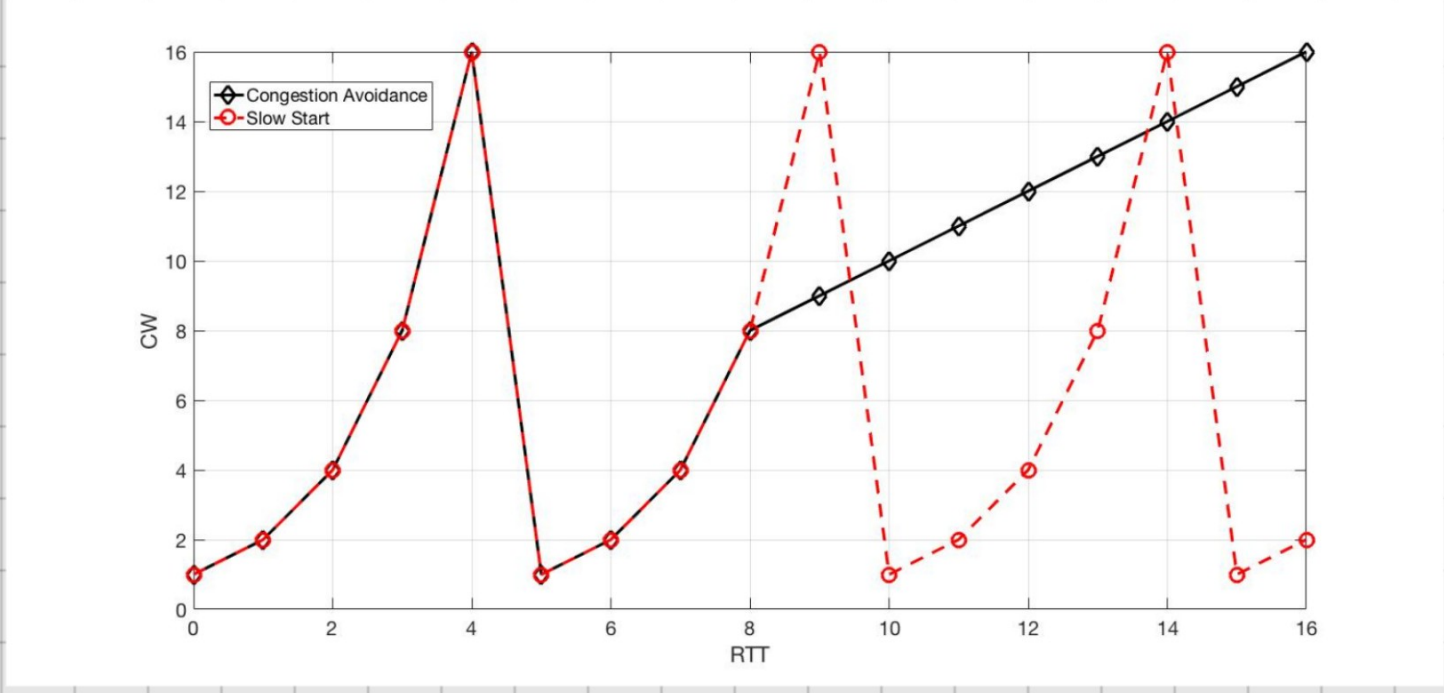
  * CONSENTIRE L'ESAURIMENTO DELLA CONGESTIONE
  * EVITARE SUCCESSIVAMENTE UN NUOVO SOVRACCARICO

- SI RIPARTE IN SLOW START SINO AL RAGGIUNGIMENTO DI UNA SOGLIA LIMITE

  ⇓

  * OLTRE LA SOGLIA LA CW CRESCE LINEARMENTE
  * LA SOGLIA È FISSATA ALLA METÀ DELLA CW DI CONGESTIONE

CONFRONTO



FAST RETRANSMIT

- MIGLIORA LE PRESTAZIONI SE È PRESO UN SINGOLO SEGMENTO

  ↓

  * VELOCIZZA LA RITRASMISSIONE DEL SEGMENTO
  * PUÓ EVITARE LA RITRASMISSIONE DEI SEGMENTI SUCCESSIVI

- ASSUNZIONI ALLA BASE

  ↓

  IL RICEVITORE EMETTE UN ACK NON APPENA RIVELA UN FUORI SEQUENZA E PER OGNI SEGMENTO SUCCESSIVO FUORI SEQUENZA

  - IN PRATICA VIENE EMESSO UN ACK OGNI VOLTA CHE VIENE RICEVUTO UN SEGMENTO
  - L'ACK FA RIFERIMENTO ALL'ULTIMO SEGMENTO RICEVUTO IN SEQUENZA

  ↓

  LA RICEZIONE DI TRE ACK DUPLICATI PUÓ ESSERE SINTOMO DI UN SEGMENTO PERSO

- LA RITRASMISSIONE DEL SEGMENTO AVVIENE ANCHE PRIMA DELLA SCADENZA DEL TIMEOUT SE SONO RICEVUTI 3 ACK DUPLICATI

  ↓

  * QUINDI AL 4° ACK DUPLICATI

- LA SCELTA DI 3 ACK DUPLICATI TENDE AD ESCLUDERE IL CASO IN CUI IL SEGMENTO SIA EFFETIVAMENTE RITARDATO IN RETE E RICEVUTO SUCCESSIVAMENTE FUORI SEQUENZA (E NON PERSO)

FAST RECOVERY

- EVITA L'INNESCO DELLA PROCEDURA STANDARD DI CONGESTION AVOIDANCE ASSOCIATA ALLA PROCEDURA DI FAST RETRANSMISSION

  ↓

* L'ARRIVO DI ACK MULTIPLI ASSICURA CHE I SEGMENTI RICEVUTI SONO STATI RICEVUTI E QUINDI L'EVENTUALE CONGESTIONE È GIÁ STATA SUPERATA

- RISPETTO ALLA PROCEDURA DI CONGESTION AVOIDANCE SI EVITA LA FASE INIZIALE DI AUMENTO ESPONENZIALE DELLA CW COME IN SLOW START

  * IL VALORE INIZIALE DELLA CW È MAGGIORE
    - VIENE FISSATO PARI ALLA SOGLIA OLTRE LA QUALE L'INCREMENTO È SEMPRE LINEARE
  * L'INCREMENTO È SEMPRE LINEARE

IMPLEMENTAZIONI DI TCP

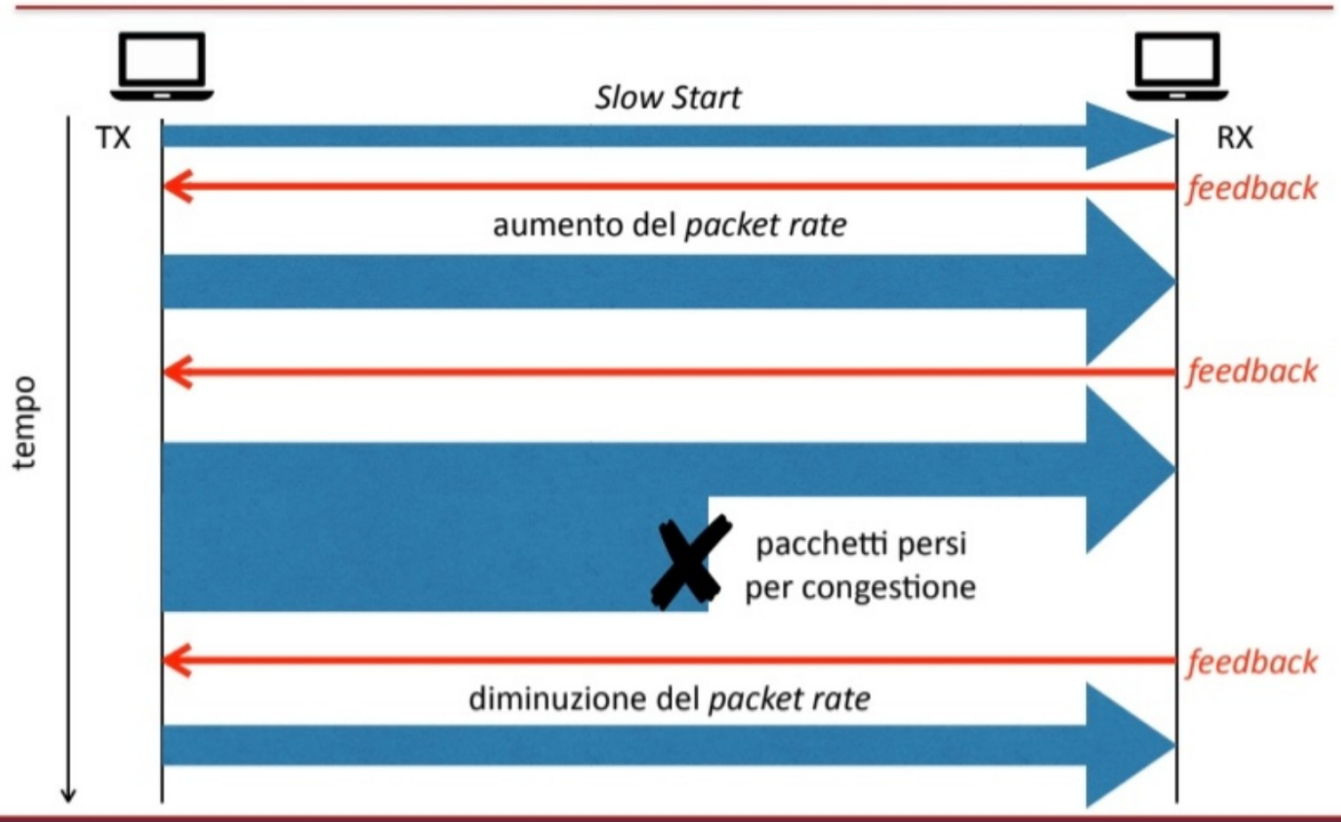
		Berkeley	Tahoe	Reno
dimensionamento del RTO	stima della varianza del RTT	X	X	X
	exponential RTO backoff	X	X	X
	algoritmo di *Karn*	X	X	X
adattamento della finestra di controllo di congestione	*Slow Start*	X	X	X
	Congestion Avoidance	X	X	X
	Fast Retransmission		X	X
	Fast recovery			X

CONFRONTO RIASSUNTIVO TRA TCP ED UDP

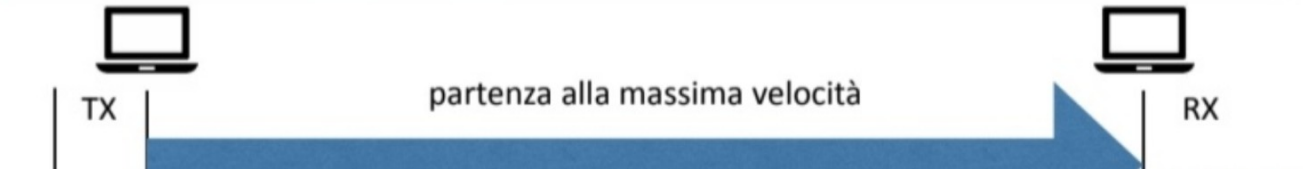
Principali differenze

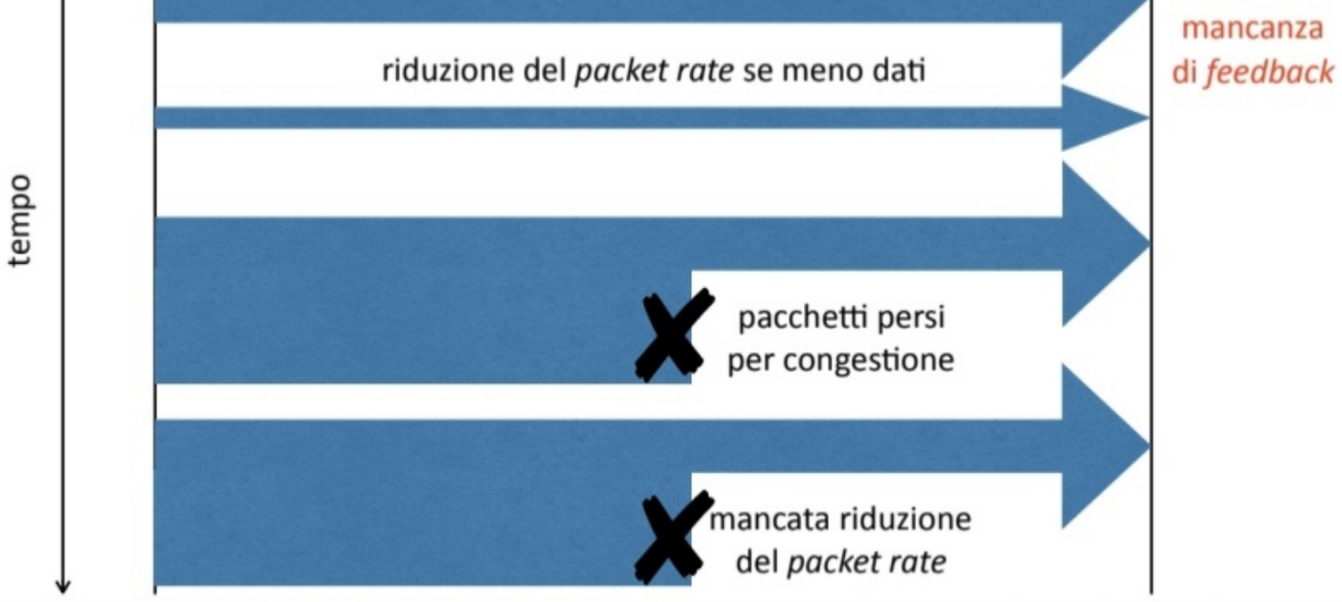
PARAMETRO	TCP	UDP
tipo di comunicazione	*stream-oriented*	*message-oriented*
paradigma di comunicazione	con connessione	senza connessione
affidabilità	SI	NO
controllo di errore/flusso/ congestione	SI	NO
overhead introdotto	maggiore	minore
supporto	esclusivamente punto-punto	anche *broadcast* e *multicast*

Trasmissione TCP



Trasmissione UDP





Confronto sulla stessa rete

